

Mitigating System Latency in Streaming Voice Conversion: An OS-Level Perspective

LI, Yudong

A Thesis Submitted in Partial Fulfilment
of the Requirements for the Degree of
Doctor of Philosophy
in
Computer Science

The Chinese University of Hong Kong, Shenzhen
April 2026

Thesis Assessment Committee

Professor CHUNG Yehching

Professor LIU Yuxuan

Abstract

The deployment of streaming voice conversion (VC) models in real-world applications is fundamentally bottlenecked by strict system-level latency requirements. While significant advancements have been made in acoustic modeling, the underlying operating system (OS) and hardware scheduling mechanisms frequently introduce non-deterministic delays. This course report investigates the intersection of streaming VC workflows and OS-level latency optimization through an in-depth review of three pivotal papers: *StreamVoice* (ICLR 2024), *Nimble* (NeurIPS 2020), and *PagedAttention* (SOSP 2023). By analyzing advanced context-aware streaming architectures alongside OS-level innovations in lightweight GPU task scheduling and dynamic virtual memory paging, this paper explores the theoretical synthesis of these concepts. We demonstrate how systems engineering is crucial for extracting the fundamental frequency and semantic features with microsecond-level predictability. This systemic stability is essential to effectively alleviate timbre leakage in real-time, zero-lookahead privacy-preserving scenarios.

Contents

Abstract	i
1 Introduction	1
2 Related Work	3
2.1 Evolution of Acoustic Modeling in VC	3
2.2 System Latency in Deep Learning Serving	4
3 Paper I: Algorithmic Foundation via StreamVoice	5
3.1 Mathematical Formulation of the Streaming Workload	5
3.2 The OS Bottleneck: Time Complexity of Reallocation	6
3.3 Contribution to the Ph.D. Thesis	7
4 Paper II: Scheduling Optimization via Nimble	8
4.1 The Anatomy of the Dispatch Bottleneck	8
4.2 Nimble’s AOT Scheduling Mechanism	9
4.3 Theoretical Application to Voice Conversion	9
5 Paper III: Memory Paging via PagedAttention	10
5.0.1 Virtual Memory and OS Paging Principles	10
5.0.2 Translating Paging to Generative AI	11
5.0.3 Theoretical Extension to Audio Streams	11

6	Synthesis of Literature for Ph.D. Research	13
6.1	Proposed System Workflow	13
6.2	Implications for Performance Evaluation	14
7	Conclusion	15
	Bibliography	16

Chapter 1

Introduction

Voice Conversion (VC) represents a critical domain in speech signal processing, aiming to modify the acoustic characteristics of a source speaker to match a target speaker while preserving the underlying linguistic content. Over the past decade, the paradigm of VC has evolved dramatically. Early approaches relying on statistical parametric synthesis (such as Gaussian Mixture Models) have been largely superseded by highly expressive deep generative models, including Generative Adversarial Networks (GANs) [1], Diffusion Models [5], and Flow Matching algorithms.

However, the transition from offline synthesis—where the entire utterance is available prior to processing—to real-time, streaming VC reveals a profound mismatch between the computational demands of deep learning algorithms and the resource management paradigms of underlying operating systems (OS). My Ph.D. research focuses on bridging this critical gap to enable high-fidelity, zero-lookahead streaming VC.

In a strict streaming context, the system is subjected to severe latency budgets. The inference pipeline must continuously ingest incoming audio chunks (often as brief as 10 to 20 milliseconds), perform feature extraction (such as esti-

inating the fundamental frequency, f_0 , and extracting high-dimensional semantic tokens), and synthesize the output waveform before the next chunk arrives. When deployed on modern OS platforms utilizing standard deep learning frameworks like PyTorch [4], these systems suffer from severe inefficiencies that manifest as unpredictable latency spikes.

Two primary system-level bottlenecks consistently emerge in this workflow:

1. **High CPU Scheduling Overhead:** Synchronous Python/C++ dispatching within deep learning frameworks overwhelms the CPU, neutralizing the benefits of GPU acceleration when operating on micro-batch sizes.
2. **Memory Fragmentation:** The continuous accumulation of historical acoustic context required for prosodic consistency leads to severe GPU memory fragmentation, forcing the OS to perform expensive, non-contiguous memory allocations and redundant tensor copying.

This report serves as a foundational literature review for my Ph.D. thesis. It critically examines three top-tier publications across the Artificial Intelligence and Computer Systems communities to understand how their isolated contributions can be theoretically synthesized to solve the streaming VC dilemma. Specifically, we review *StreamVoice* [8] to define the algorithmic workload, *Nimble* [3] to address task scheduling overheads, and *PagedAttention* [2] to explore virtual memory paging for context management. By dissecting these works, this paper outlines how algorithm-system co-design forms the theoretical backbone of my ongoing doctoral research.

Chapter 2

Related Work

The challenge of real-time voice conversion sits uniquely at the nexus of acoustic signal processing and computer systems engineering. To contextualize the reviewed papers, it is necessary to examine the historical trajectory of both fields.

2.1 Evolution of Acoustic Modeling in VC

Traditional VC frameworks, such as those based on joint density Gaussian Mixture Models (GMMs) [6] or early sequence-to-sequence bidirectional Recurrent Neural Networks (RNNs), operate on complete utterances. These models inherently utilize global variance and bidirectional dependencies, making them fundamentally incompatible with streaming applications.

Recent algorithmic advancements have shifted towards causal, chunk-based processing. Architectures utilizing causal dilated convolutions (e.g., WaveNet-based variants [7]) and unidirectional Transformers allow for inference by processing audio in discrete blocks. However, to maintain prosodic consistency and acoustic naturalness, these models must carry over hidden states or explicit feature caches from previous chunks. While researchers strive to optimize the math-

emational operations within these neural layers—proposing linear attention mechanisms or state-space models—they frequently treat the underlying hardware as an abstract, infinite-bandwidth resource. As the duration of the streaming session increases, the continuous reallocation of memory for the growing context cache drastically degrades the system’s Input/Output (I/O) performance.

2.2 System Latency in Deep Learning Serving

Within the computer systems and OS communities, the optimization of deep learning inference has historically focused on throughput-oriented serving platforms, such as NVIDIA Triton Inference Server or ONNX Runtime. These systems achieve high performance by dynamically batching multiple independent requests to maximize GPU utilization and memory bandwidth.

However, these throughput-oriented solutions are ill-suited for the strict, single-stream, microsecond-level latency requirements of zero-lookahead VC. Standard deep learning frameworks introduce a significant “framework tax.” Dynamic computation graphs require the CPU to interpret the graph topology, allocate temporary memory spaces, and dispatch CUDA kernels to the GPU for every single forward pass [4]. In a streaming VC scenario where operations are performed on micro-chunks, the CPU scheduling time often exceeds the actual GPU computation time. While techniques like model quantization (e.g., INT8/FP8) and operator fusion reduce raw computation time, they fail to resolve the structural overhead of OS-level process scheduling.

Chapter 3

Paper I: Algorithmic Foundation via StreamVoice

To formally define the system-level interventions required, we must first mathematically define the computational workload. We achieve this by reviewing *StreamVoice: Streamable Context-Aware Language Modeling for Real-time Zero-Shot Voice Conversion* [8].

3.1 Mathematical Formulation of the Streaming Workload

StreamVoice addresses the primary acoustic challenge of streaming generation: processing speech chunk-by-chunk without future context (lookahead) typically yields severe acoustic artifacts and jitter. Let the continuous input source speech be divided into a sequence of micro-chunks $\mathbf{X} = \{x_1, x_2, \dots, x_T\}$, where each chunk x_t spans Δt milliseconds.

For a given chunk x_t , the feature extraction pipeline must isolate the linguistic content c_t and the fundamental frequency f_t . In a memoryless, purely Markovian

system, the generative model G_θ would synthesize the output y_t solely based on current features and a target speaker embedding s :

$$y_t = G_\theta(c_t, f_t, s) \quad (3.1)$$

However, to preserve prosodic continuity, *StreamVoice* introduces a continuous context cache, denoted as $\mathbf{H}_t$, which aggregates acoustic representations from all preceding time steps. The streaming generation process is thus formally defined as:

$$y_t = G_\theta(c_t, f_t, s, \mathbf{H}_{t-1}) \quad (3.2)$$

where the cache is recursively updated:

$$\mathbf{H}_t = \text{Concat}(\mathbf{H}_{t-1}, h_t) \quad (3.3)$$

with h_t representing the intermediate neural features of the current chunk x_t .

3.2 The OS Bottleneck: Time Complexity of Reallocation

While Equation 3.2 ensures high acoustic fidelity algorithmically, Equation 3.3 introduces a critical structural bottleneck. In modern OS and GPU architectures, multi-dimensional tensors require contiguous physical memory spaces to maximize Single Instruction, Multiple Data (SIMD) execution efficiency.

As the streaming session progresses, the context cache $\mathbf{H}_t$ grows linearly. The OS memory allocator (e.g., `cudaMalloc`) cannot simply append data if the adjacent virtual memory pages are mapped to other physical addresses or occupied by other processes. Consequently, the concatenation in Equation 3.3 triggers a catastrophic sequence of system calls:

1. **Allocation:** The OS allocates a new contiguous block of size $|\mathbf{H}_{t-1}| + |h_t|$.

2. **Data Transfer:** The entire historical tensor $\mathbf{H}_{t-1}$ is copied into the new block via memory bus I/O.
3. **Deallocation:** The original memory block is freed, resulting in external memory fragmentation.

This process possesses an asymptotic time complexity of $O(t)$ per chunk. Over a session of length T , the total memory bandwidth cost scales to $O(T^2)$. This algorithmic necessity dictates that the OS will eventually fail to meet the 20ms deadline, causing audio dropouts.

3.3 Contribution to the Ph.D. Thesis

The analysis of *StreamVoice* is foundational to my thesis. It proves that the barrier to deploying high-fidelity streaming VC is a classic memory management problem rather than a purely acoustic limitation. This directs my research toward identifying OS-level interventions capable of managing theoretically infinite context lengths without triggering $O(T^2)$ memory copying overheads.

Chapter 4

Paper II: Scheduling

Optimization via Nimble

Having defined the memory constraint, we must address the compute constraint: the CPU-bound scheduling latency during concurrent feature extraction. For this, we review *Nimble: Lightweight and Parallel GPU Task Scheduling for Deep Learning* [3].

4.1 The Anatomy of the Dispatch Bottleneck

In a streaming VC pipeline, estimating the fundamental frequency (f_0) and extracting semantic tokens (via models like HuBERT) must occur simultaneously to meet strict deadlines. However, the execution engine of frameworks like PyTorch introduces a severe bottleneck.

Let T_{total} be the total processing time for an audio chunk x_t . It is composed of the CPU dispatch time T_{cpu} and the GPU execution time T_{gpu} :

$$T_{total} = T_{cpu_dispatch} + T_{gpu_compute} \quad (4.1)$$

During offline batch processing, the massive parallel workload ensures $T_{gpu_compute} \gg$

$T_{cpu_dispatch}$. However, in zero-lookahead streaming (batch size = 1, sequence length = 10ms), the GPU computation finishes in microseconds. $T_{cpu_dispatch}$ —which involves Python Global Interpreter Lock (GIL) contention, C++ bindings, and sequential CUDA kernel launches—becomes the dominant factor. The GPU sits idle while the CPU struggles to schedule the next micro-task.

4.2 Nimble’s AOT Scheduling Mechanism

Nimble tackles this inherent inefficiency by bypassing the dynamic, synchronous dispatch overhead entirely. It introduces an Ahead-of-Time (AOT) scheduling mechanism [3]. *Nimble* performs a ”warm-up” pass to capture the entire execution trace of a model into a static, topological graph. Memory allocations are pre-determined, and synchronization points are explicitly mapped.

Crucially, by analyzing data dependencies within the computational graph, *Nimble* maps independent operations to distinct, parallel CUDA streams. During live inference, the CPU issues a single, lightweight system call to launch the pre-compiled graph, effectively reducing $T_{cpu_dispatch}$ to near zero.

4.3 Theoretical Application to Voice Conversion

Applying *Nimble*’s methodology to VC theoretically resolves the extraction bottleneck. By capturing the f_0 extraction network and the semantic encoder into a unified static graph, the system can execute these independent feature extractions concurrently on parallel GPU streams. This OS-level scheduling intervention guarantees deterministic execution times, ensuring that the feature extraction phase reliably completes well within the Δt deadline, providing the necessary computational headroom for the subsequent generative synthesis.

Chapter 5

Paper III: Memory Paging via PagedAttention

With CPU scheduling optimized, we return to the $O(T^2)$ memory reallocation disaster identified in Chapter 3. We review *Efficient Memory Management for Large Language Model Serving with PagedAttention* [2] to explore how classic OS memory concepts solve this in AI serving.

5.1 Virtual Memory and OS Paging

In traditional OS design, physical memory fragmentation is mitigated through Virtual Memory. When a process requests a large contiguous block, the OS provides a contiguous virtual address space but maps it to non-contiguous physical memory pages via a Page Table and the Translation Lookaside Buffer (TLB). This mechanism avoids moving physical data when memory pools expand.

5.2 Translating Paging to Generative AI

PagedAttention applies this OS concept to the user-space memory management of Large Language Models [2]. Instead of storing the Keys and Values (KV-cache) of historical tokens in a contiguous tensor, it partitions the cache into fixed-size "blocks" (analogous to memory pages) and manages them via a Block Table.

During generation, the attention mechanism computes the correlation between the current query q_t and historical keys $\mathbf{K}$ sequentially across discrete memory pages, rather than requiring a unified contiguous matrix:

$$\text{Attention}(q_t, \mathbf{K}, \mathbf{V}) = \sum_{i \in \text{BlockTable}} \text{AttnOp}(q_t, K_{P_i}, V_{P_i}) \quad (5.1)$$

5.3 Theoretical Extension to Audio Streams

The theoretical extrapolation of *PagedAttention* from discrete text tokens to continuous audio streams forms the crux of my Ph.D. memory management strategy. In a streaming VC context, an "audio page" can be defined as a memory block storing the continuous acoustic features (h_t) for a fixed number of audio frames.

When a new audio chunk arrives and the cache must be updated, the system simply allocates a new physical page from a pre-allocated pool and updates the Block Table. No historical data $\mathbf{H}_{t-1}$ is ever copied. This OS-inspired intervention theoretically reduces the context update complexity from $O(T)$ to $O(1)$, completely eliminating external memory fragmentation and the associated latency spikes.

Furthermore, managing audio context in discrete pages provides a critical privacy benefit. In scenarios requiring strict speaker anonymization, raw pre-conversion acoustic features can be isolated into secure memory pages with strict access control, heavily alleviating the risk of timbre leakage through unauthorized memory dumps.

Chapter 6

Synthesis of Literature for Ph.D. Research

The critical review of these three papers provides a comprehensive, theoretically sound blueprint for my doctoral research. The synthesis of these methodologies yields a unified strategy for zero-lookahead streaming VC.

6.1 Proposed System Workflow

Based on the literature, the optimal theoretical dataflow for a latency-critical VC system is as follows:

1. **Ingestion:** An audio chunk x_t arrives.
2. **Parallel Extraction:** Leveraging the AOT scheduling principles from *Nimble*, the system launches a static graph, performing concurrent extraction of f_0 and semantic tokens on parallel CUDA streams with zero CPU dispatch overhead.
3. **Paging Context Update:** The extracted features h_t are written to the

current active "audio page" utilizing the mechanics derived from *PagedAttention*. If the page is full, a new physical block is mapped in $O(1)$ time.

4. **Attention and Generation:** The generative backbone computes attention across the Block Table, synthesizing the output waveform y_t conditioned on the paged historical context and current features.

6.2 Implications for Performance Evaluation

This synthesis dictates a dual-faceted evaluation methodology for my future research. Traditional acoustic metrics, such as Word Error Rate (WER) and Subjective Mean Opinion Score (SMOS), remain necessary to validate the generative quality established by the *StreamVoice* baseline [8].

However, evaluating the efficacy of the system-level interventions requires rigorous OS profiling. Future experiments will require profiling tools like NVIDIA Nsight Systems to visualize execution timelines, confirming the overlap of kernel execution and the reduction of CPU idle time. Most importantly, evaluating the success of the paging mechanism will rely on tracking the P99 tail latency and demonstrating a flat, $O(1)$ memory allocation overhead curve across indefinitely long streaming sessions.

Chapter 7

Conclusion

The demand for high-fidelity, zero-lookahead streaming voice conversion exposes the fundamental limitations of modern deep learning frameworks when deployed on standard operating systems. This literature review has critically examined *StreamVoice*, *Nimble*, and *PagedAttention* to demonstrate that the barriers to real-time deployment are deeply rooted in classic OS resource management challenges: CPU scheduling and contiguous memory allocation.

By dissecting the algorithmic requirement of continuous context caching, we identified the source of $O(T^2)$ memory bandwidth degradation. The reviewed literature strongly suggests that bridging acoustic algorithms with systems engineering—specifically through Ahead-of-Time parallel task scheduling and user-space virtual memory paging—is theoretically viable and necessary. The synthesis of these concepts forms the foundational hypothesis of my Ph.D. thesis: that algorithm-system co-design is the key to unlocking deterministic, memory-efficient streaming environments, ultimately providing the systemic stability required for voice conversion in real-world deployments.

Bibliography

- [1] Hirokazu Kameoka, Takuhiro Kaneko, Kou Tanaka, and Nobukatsu Hojo. Stargan-vc: Non-parallel many-to-many voice conversion using star generative adversarial networks. In *2018 IEEE Spoken Language Technology Workshop (SLT)*, pages 266–273. IEEE, 2018. [1](#)
- [2] Woosuk Kwon, Zhuohan Li, Siyuan Zhuang, Ying Sheng, Lianmin Zheng, Cody Hao Yu, Joseph Gonzalez, Hao Zhang, and Ion Stoica. Efficient memory management for large language model serving with pagedattention. In *Proceedings of the 29th symposium on operating systems principles*, pages 611–626, 2023. [2](#)
- [3] Woosuk Kwon, Gyeong-In Yu, Eunji Jeong, and Byung-Gon Chun. Nimble: Lightweight and parallel gpu task scheduling for deep learning. *Advances in Neural Information Processing Systems*, 33:8343–8354, 2020. [2](#), [8](#), [9](#)
- [4] Adam Paszke, Sam Gross, Francisco Massa, Adam Lerer, James Bradbury, Gregory Chanan, Trevor Killeen, Zeming Lin, Natalia Gimelshein, Luca Antiga, et al. Pytorch: An imperative style, high-performance deep learning library. *Advances in neural information processing systems*, 32, 2019. [2](#), [4](#)
- [5] Vadim Popov, Ivan Vovk, Vladimir Gogoryan, Tasnima Sadekova, and

- Mikhail Kudinov. Grad-tts: A diffusion probabilistic model for text-to-speech. In *International conference on machine learning*, pages 8599–8608. PMLR, 2021. [1](#)
- [6] Tomoki Toda, Alan W Black, and Keiichi Tokuda. Voice conversion based on maximum-likelihood estimation of spectral parameter trajectory. *IEEE Transactions on Audio, Speech, and Language Processing*, 15(8):2222–2235, 2007. [3](#)
- [7] Aäron van den Oord, Sander Dieleman, Heiga Zen, Karen Simonyan, Oriol Vinyals, Alex Graves, Nal Kalchbrenner, Andrew Senior, and Koray Kavukcuoglu. Wavenet: A generative model for raw audio. In *Proc. SSW 2016*, pages 125–125, 2016. [3](#)
- [8] Zhichao Wang, Yuanzhe Chen, Xinsheng Wang, Lei Xie, and Yuping Wang. Streamvoice: Streamable context-aware language modeling for real-time zero-shot voice conversion. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 7328–7338, 2024. [2](#), [5](#), [14](#)